b_07: バックオフィス基盤設計 — マイナンバー管理・RLS職務分離・フロントエンド7モジュール

> **Source of Truth**: 本ドキュメント
> **対象フェーズ**: Phase 2+（バックオフィスモジュール ¥200,000/月）
> **前提**: `a_01_セキュリティ設計.md` §10(PII), §11(RBAC), §12(暗号化) を踏襲
> **依存**: `security/encryption.py` （AES-256-GCM実装済み）、 `db/schema.sql` （既存12テーブル）

目次

1. 項目8: マイナンバー管理ライフサイクル設計
2. 項目9: RLS・職務分離ポリシー設計
3. 項目10: フロントエンド7モジュールUI設計概要

項目8: マイナンバー管理ライフサイクル設計

> 個人番号（マイナンバー）は番号法（行政手続における特定の個人を識別するための番号の利用等に関する法律）に基づき、
> 収集・保管・利用・廃棄の全ライフサイクルで厳格な管理が求められる。
> 本設計は、特定個人情報の適正な取扱いに関するガイドライン（事業者編）に準拠する。

8.1 全体ライフサイクル図

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│  収集    │──▶│  保管    │──▶│  利用    │──▶│  廃棄    │
│Collection│    │ Storage  │    │ Usage    │    │ Disposal │
└──────────┘    └──────────┘    └──────────┘    └──────────┘
     │               │               │               │
     ▼               ▼               ▼               ▼
本人確認書類      AES-256-GCM     源泉徴収票作成    NULLクリア
同意取得          アクセス制御    社保届出          廃棄ログ記録
目的明示          アクセスログ    支払調書作成      ログ永久保存
```

8.2 収集フェーズ

8.2.1 本人確認書類の種類

マイナンバーの収集時には「番号確認」と「身元確認」の2段階の本人確認が必要。

確認種別	書類	備考
番号確認（以下いずれか1点）	マイナンバーカード（表面）	番号確認+身元確認を1枚で完了可
	通知カード	2020年5月以降は新規発行なし。既発行分は有効
	マイナンバー記載の住民票の写し	発行後3ヶ月以内
身元確認（以下いずれか1点）	マイナンバーカード（裏面の顔写真）	番号確認と併用時はこれ1枚で完結
	運転免許証	顔写真付き
	パスポート	顔写真付き
	在留カード	外国人従業員向け
	顔写真なしの場合	健康保険証 + 年金手帳（2点セット）

システム上の管理方法:

```
my_number_identity_docs テーブル:
  - id: UUID
  - employee_id: UUID FK
  - company_id: UUID FK（RLS用）
  - doc_type: ENUM('my_number_card', 'notification_card', 'resident_cert',
                   'drivers_license', 'passport', 'residence_card',
                   'health_insurance', 'pension_book')
  - doc_category: ENUM('number_verification', 'identity_verification')
  - verified_by: UUID FK（確認者のuser_id）
  - verified_at: TIMESTAMPTZ
  - file_storage_ref: TEXT（Supabase Storage暗号化バケット参照。原本画像は保存しない方針も可）
  - notes: TEXT
  - created_at: TIMESTAMPTZ
```

8.2.2 収集時のUI設計要件

要件	実装方法	理由
SSL/TLS必須	フロントエンドは HTTPS のみで配信。HTTP アクセスは 301 リダイレクト	通信経路上での番号傍受を防止
入力後即暗号化	フロントエンドで入力 → API送信 → サーバー側で即 `encrypt_field()` → DB保存。平文はメモリ上のみ	DBに平文マイナンバーを一切保存しない
画面に平文表示しない	入力フォームは `type="password"` で表示。確認画面では `****-****-1234`（下4桁のみ）	画面の覗き見・スクリーンショットによる漏洩防止
コピー&ペースト無効化	`onCopy`, `onPaste` イベントを無効化（入力フォーム）	クリップボード経由の漏洩防止
ブラウザ自動保存禁止	`autocomplete="off"` を設定	ブラウザのパスワードマネージャーへの保存を防止
セッションタイムアウト	マイナンバー入力画面は5分間操作なしで自動ログアウト	離席時のリスク低減
入力画面のアクセス制限	`my_number_admin` ロールのみ入力画面にアクセス可	権限のないユーザーが入力画面を開けない

入力フォーム仕様:

```
[マイナンバー入力フォーム]

┌──────────────────────────────────────┐
│  従業員のマイナンバーを登録する                │
│                                        │
│  従業員名:  山田 太郎                      │
│  収集目的:  ☑ 源泉徴収票の作成              │
│             ☑ 社会保険届出                  │
│             □ 支払調書の作成                │
│                                        │
│  マイナンバー: [●●●●●●●●●●●●]              │
│  (12桁の数字を入力してください)              │
│                                        │
│  確認のため再入力: [●●●●●●●●●●●●]           │
│                                        │
│  ☑ 番号確認書類を確認しました               │
│  ☑ 身元確認書類を確認しました               │
│  ☑ 本人から収集目的について同意を得ました     │
│                                        │
│  [マイナンバーを登録する]                   │
└──────────────────────────────────────┘
```

バリデーション:

- 12桁の数字のみ許可（ハイフン自動除去）
- チェックデジット検証（個人番号の検査用数字アルゴリズム）
- 2回入力の一致確認
- 全チェックボックスの選択必須

8.2.3 収集目的の明示

番号法で認められた利用目的のみを選択肢として提示する。

利用目的コード	利用目的	法的根拠	対象書類
`withholding_tax`	源泉徴収票の作成	所得税法第226条	給与所得の源泉徴収票
`social_insurance`	社会保険届出	厚生年金保険法・健康保険法	被保険者資格取得届等
`payment_report`	支払調書の作成	所得税法第225条	報酬、料金、契約金等の支払調書

収集目的の提示ルール:

- 従業員 → `withholding_tax` + `social_insurance` を必須表示
- 外注先・個人事業主 → `payment_report` を表示
- 目的外利用は一切不可（UIでも選択肢を出さない）

8.2.4 同意取得の記録

```
my_number_consents テーブル:
  - id: UUID PK
  - company_id: UUID FK（RLS用）
  - employee_id: UUID FK
  - purpose_codes: TEXT[] （['withholding_tax', 'social_insurance'] 等）
  - consented_at: TIMESTAMPTZ NOT NULL
  - consent_method: ENUM('system_form', 'paper_scan', 'verbal_record')
  - collected_by: UUID FK（収集担当者のuser_id）
  - ip_address: INET
  - user_agent: TEXT
  - paper_document_ref: TEXT（紙の同意書をスキャンした場合のStorage参照）
  - revoked_at: TIMESTAMPTZ（撤回された場合）
  - created_at: TIMESTAMPTZ
```

同意取得フロー:

```
1. 収集担当者が入力画面を開く
2. 従業員を選択
3. 収集目的チェックボックスを選択（最低1つ必須）
4. 「本人から収集目的について同意を得ました」にチェック
5. マイナンバーを入力
6. [マイナンバーを登録する] ボタン押下
7. → my_number_consents にレコード挿入
8. → my_numbers テーブルに暗号化して保存
9. → my_number_access_log に「収集」ログを記録
10. → audit_logs にも記録
```

8.3 保管フェーズ

8.3.1 暗号化方式

項目	仕様
アルゴリズム	AES-256-GCM（認証付き暗号化）
鍵管理（MVP）	環境変数 `MY_NUMBER_ENCRYPTION_KEY`（base64エンコード、32バイト）。`ENCRYPTION_KEY` とは別の鍵を使用
鍵管理（本番）	GCP KMS `projects/{project}/locations/asia-northeast1/keyRings/shachotwo/cryptoKeys/my-number-key`
nonce	96ビット（12バイト）、各暗号化ごとにランダム生成
鍵ローテーション	年1回。旧鍵で復号 → 新鍵で再暗号化のバッチ処理
実装	`security/encryption.py` の `encrypt_field()` / `decrypt_field()` を拡張。鍵セレクタを追加

鍵分離の理由:
マイナンバーは一般PIIより高い保護水準が必要。鍵を分離することで、一般PII鍵の漏洩がマイナンバーの漏洩に直結しない。

encryption.py 拡張案:

```
# 鍵の種別
class KeyPurpose(str, Enum):
    GENERAL_PII = "general_pii"        # 住所・電話番号等
    MY_NUMBER = "my_number"            # マイナンバー専用
    FINANCIAL = "financial"            # 口座番号等

def _get_key(purpose: KeyPurpose = KeyPurpose.GENERAL_PII) -> bytes:
    env_map = {
        KeyPurpose.GENERAL_PII: "ENCRYPTION_KEY",
        KeyPurpose.MY_NUMBER: "MY_NUMBER_ENCRYPTION_KEY",
        KeyPurpose.FINANCIAL: "FINANCIAL_ENCRYPTION_KEY",
    }
    key_b64 = os.environ.get(env_map[purpose])
    ...
```

8.3.2 データベース設計

```
-- マイナンバー保管テーブル
CREATE TABLE my_numbers (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    company_id UUID NOT NULL REFERENCES companies(id),
    employee_id UUID NOT NULL REFERENCES employees(id),
    encrypted_number TEXT NOT NULL,         -- AES-256-GCM暗号化済み
    number_hash TEXT NOT NULL,              -- SHA-256ハッシュ（重複チェック用。ソルト付き）
    purpose_codes TEXT[] NOT NULL,          -- ['withholding_tax', 'social_insurance']
    collected_at TIMESTAMPTZ NOT NULL,
    collected_by UUID NOT NULL REFERENCES users(id),
    status TEXT NOT NULL DEFAULT 'active'
        CHECK (status IN ('active', 'suspended', 'disposed')),
    disposed_at TIMESTAMPTZ,
    disposed_by UUID REFERENCES users(id),
    disposal_reason TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(company_id, employee_id)        -- 1従業員1レコード
);

-- RLSは有効化するが、ポリシーはアプリ層で制御（後述）
ALTER TABLE my_numbers ENABLE ROW LEVEL SECURITY;

-- デフォルトポリシー: サービスロールのみアクセス可（RLSを通さずアプリ層で制御）
CREATE POLICY "my_numbers_service_only" ON my_numbers
    USING (false)  -- 通常ユーザーは一切アクセス不可
    WITH CHECK (false);
```

8.3.3 アクセス制御

RLSではなくアプリ層で制御する理由:

- マイナンバーは `company_id` によるテナント分離だけでは不十分
- 同一テナント内でも、復号権限者（`my_number_admin`）以外はアクセス不可
- RLSのポリシーではロール判定が複雑になるため、アプリ層で明示的に制御する

アクセス制御マトリクス:

操作	`my_number_admin`	`hr_manager`	`finance`	`admin`	`employee`
マイナンバー登録（暗号化保存）	OK	NG	NG	NG	NG
マイナンバー復号（平文取得）	OK	NG	NG	NG	NG
マイナンバーマスク表示（下4桁）	OK	OK	NG	NG	NG
アクセスログ閲覧	OK	NG	NG	OK	NG
廃棄実行	OK	NG	NG	NG	NG

アプリ層での制御実装方針:

```
# routers/hr.py または専用の routers/my_number.py
from auth.middleware import require_role

@router.post("/employees/{employee_id}/my-number")
async def register_my_number(
    employee_id: UUID,
    request: MyNumberRegisterRequest,
    user: JWTClaims = Depends(require_role("my_number_admin")),
):
    # 1. バリデーション（12桁、チェックデジット）
    # 2. 同意レコード確認
    # 3. encrypt_field(number, purpose=KeyPurpose.MY_NUMBER)
    # 4. DB保存（サービスロールクライアント使用）
    # 5. アクセスログ記録
    ...
```

8.3.4 アクセスログ

```
CREATE TABLE my_number_access_log (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    company_id UUID NOT NULL,
    employee_id UUID NOT NULL,             -- 対象従業員
    accessed_by UUID NOT NULL,             -- アクセスしたユーザー
    access_type TEXT NOT NULL CHECK (access_type IN (
        'collect',       -- 収集（初回登録）
        'view_masked',   -- マスク表示（下4桁）
        'view_full',     -- 復号（平文取得）
        'export',        -- 帳票出力（源泉徴収票等）
        'update',        -- 更新（番号変更）
        'dispose'        -- 廃棄
    )),
    purpose TEXT NOT NULL,                 -- 'withholding_tax_2025' 等の具体的目的
    ip_address INET NOT NULL,
    user_agent TEXT,
    details JSONB,                         -- 追加情報（出力帳票名等）
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- このテーブルはINSERT ONLYポリシー（削除・更新不可）
ALTER TABLE my_number_access_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY "my_number_access_log_insert_only" ON my_number_access_log
    FOR INSERT WITH CHECK (true);
CREATE POLICY "my_number_access_log_select_admin" ON my_number_access_log
    FOR SELECT USING (
        company_id = (current_setting('app.company_id', true))::UUID
    );
-- UPDATE/DELETE ポリシーなし → 物理的に不可

-- インデックス
CREATE INDEX idx_my_number_access_log_company ON my_number_access_log (company_id,
created_at DESC);
CREATE INDEX idx_my_number_access_log_employee ON my_number_access_log (employee_id,
created_at DESC);
CREATE INDEX idx_my_number_access_log_user ON my_number_access_log (accessed_by,
created_at DESC);
```

8.3.5 物理的安全管理措置（Cloud Run環境）

措置	実装
メモリ内復号	復号はCloud Run上のリクエスト処理中のみ。レスポンス返却後、Pythonのガベージコレクタが平文を回収。明示的に `del plaintext` + `gc.collect()` を実行
ディスク書き込み禁止	Cloud Runはステートレス。`/tmp` への平文書き出し禁止。ログに平文を出力しない
コンテナ分離	Cloud Runの各リクエストはgVisorサンドボックス内で実行。コンテナ間のメモリ共有なし
ネットワーク分離	マイナンバー関連APIはVPCコネクタ経由のみ。パブリックインターネットから直接アクセス不可（将来要件）
スワップ無効化	Cloud Runはスワップを使用しない設計。メモリ上の平文がディスクに書き出されるリスクなし

8.4 利用フェーズ

8.4.1 利用目的の制限

マイナンバーの利用は以下の3目的に**厳格に限定**する。これ以外の目的での復号は技術的に阻止する。

利用目的	呼び出し元	復号が必要か
源泉徴収票の作成	`workers/bpo/backoffice/withholding_tax_pipeline.py`	YES（帳票に番号を記載）
社会保険届出	`workers/bpo/backoffice/social_insurance_pipeline.py`	YES（届出書に番号を記載）
支払調書の作成	`workers/bpo/backoffice/payment_report_pipeline.py`	YES（調書に番号を記載）

技術的制限の実装:

```
ALLOWED_DECRYPT_PURPOSES = frozenset([
    "withholding_tax",
    "social_insurance",
    "payment_report",
])

async def decrypt_my_number(
    employee_id: UUID,
    purpose: str,
    user: JWTClaims,
    ip_address: str,
) -> str:
    """マイナンバーを復号する。利用目的チェック+ログ記録を強制。"""
    # 1. 目的チェック
    if purpose not in ALLOWED_DECRYPT_PURPOSES:
        raise PermissionError(f"マイナンバーの利用目的が不正です: {purpose}")

    # 2. ロールチェック
    if "my_number_admin" not in user.roles:
        raise PermissionError("マイナンバーの復号権限がありません")

    # 3. 復号
    record = await _fetch_encrypted_my_number(employee_id, user.company_id)
    plaintext = decrypt_field(record.encrypted_number, purpose=KeyPurpose.MY_NUMBER)

    # 4. アクセスログ記録（復号前にログを書くことで、ログ書き込み失敗時は復号しない）
    await _log_my_number_access(
        company_id=user.company_id,
        employee_id=employee_id,
        accessed_by=user.user_id,
        access_type="view_full",
        purpose=purpose,
        ip_address=ip_address,
    )

    return plaintext
```

8.4.2 利用時のログ記録

全ての復号操作は `my_number_access_log` テーブルに記録する。ログは以下の情報を含む:

フィールド	内容	例
`accessed_by`	操作者のuser_id	`550e8400-e29b-...`
`access_type`	操作種別	`view_full`
`purpose`	具体的利用目的	`withholding_tax_2025`
`ip_address`	操作元IPアドレス	`203.0.113.50`
`details`	追加情報	`{"output": "withholding_tax_slip_2025.pdf"}`
`created_at`	操作日時	`2026-03-28T10:30:00+09:00`

ログ記録のタイミング:
- 復号リクエスト受信時（復号実行前）にログを書く
- ログ書き込みに失敗した場合、復号処理自体を中止する（fail-closed）

8.4.3 画面表示時のマスキング

画面	表示形式	対象ロール
従業員一覧	番号の有無のみ表示（「登録済み」/「未登録」）	`hr_manager`, `my_number_admin`
従業員詳細	`****-****-1234`（下4桁のみ）	`my_number_admin`
帳票プレビュー	`****-****-1234`（下4桁のみ）	`my_number_admin`
帳票出力（PDF）	完全な番号（12桁）	システム処理（画面表示なし）
上記以外の全画面	非表示（フィールド自体を表示しない）	全ロール

マスキング実装:

```
def mask_my_number(number: str) -> str:
    """マイナンバーをマスクして下4桁のみ表示。"""
    if len(number) != 12:
        return "****-****-****"
    return f"****-****-{number[-4:]}"
```

8.5 廃棄フェーズ

8.5.1 廃棄条件

マイナンバーの廃棄は、利用目的に対応する**法定保存期間の経過後**に実施する。

利用目的	法定保存期間	起算日	根拠法令
源泉徴収票	**7年**	法定申告期限の翌日	所得税法施行規則第76条の3
社会保険届出	**2年**	届出完了日	厚生年金保険法施行規則第28条
支払調書	**7年**	法定申告期限の翌日	所得税法施行規則第83条

廃棄タイミングの決定ルール:

- 複数の利用目的がある場合、**最も長い保存期間**を採用する
- 例: 源泉徴収票（7年）+ 社会保険届出（2年）→ 7年を採用
- 退職日ではなく、最後の利用日（帳票作成日）から起算する

```
廃棄可能日の計算:
  最終利用日 = MAX(各利用目的の最終帳票作成日)
  最長保存期間 = MAX(各利用目的の法定保存期間)
  廃棄可能日 = 最終利用日 + 最長保存期間
```

8.5.2 廃棄方法

```
-- 廃棄処理（トランザクション内で実行）
BEGIN;

-- 1. 暗号化カラムをNULLに更新（暗号文自体を消去）
UPDATE my_numbers
SET
    encrypted_number = NULL,
    number_hash = NULL,
    status = 'disposed',
    disposed_at = NOW(),
    disposed_by = :user_id,
    disposal_reason = :reason
WHERE id = :my_number_id
  AND company_id = :company_id
  AND status = 'active';

-- 2. 廃棄ログの記録
INSERT INTO my_number_disposal_log (
    id, company_id, employee_id, disposed_by,
    disposal_reason, retention_period_years,
    original_collected_at, last_used_at,
    created_at
) VALUES (
    uuid_generate_v4(), :company_id, :employee_id, :user_id,
    :reason, :retention_years,
    :collected_at, :last_used_at,
    NOW()
);

-- 3. アクセスログに廃棄を記録
INSERT INTO my_number_access_log (
    id, company_id, employee_id, accessed_by,
    access_type, purpose, ip_address, details,
    created_at
) VALUES (
    uuid_generate_v4(), :company_id, :employee_id, :user_id,
    'dispose', '法定保存期間経過による廃棄', :ip_address,
    jsonb_build_object('retention_years', :retention_years, 'reason', :reason),
    NOW()
);

COMMIT;
```

8.5.3 廃棄ログの保存

```
CREATE TABLE my_number_disposal_log (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    company_id UUID NOT NULL,
    employee_id UUID NOT NULL,
    disposed_by UUID NOT NULL,
    disposal_reason TEXT NOT NULL,
    retention_period_years INT NOT NULL,
    original_collected_at TIMESTAMPTZ NOT NULL,
    last_used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
    -- ★ updated_at は意図的に設けない（変更不可）
);

-- INSERT ONLYポリシー（廃棄ログ自体は永久保存・削除不可）
ALTER TABLE my_number_disposal_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY "disposal_log_insert_only" ON my_number_disposal_log
    FOR INSERT WITH CHECK (true);
CREATE POLICY "disposal_log_select" ON my_number_disposal_log
    FOR SELECT USING (
        company_id = (current_setting('app.company_id', true))::UUID
    );
-- UPDATE/DELETE ポリシーなし → 物理的に不可

CREATE INDEX idx_disposal_log_company ON my_number_disposal_log (company_id, created_at
DESC);
```

廃棄ログの保存期間: 永久保存

- 廃棄ログは「マイナンバーを適切に廃棄した」ことの証拠。税務調査・監査で求められる
- TTL設定なし。テーブルサイズは極めて小さい（従業員数 x 1レコード程度）

8.5.4 自動廃棄スケジューラ

```
# workers/bpo/backoffice/my_number_disposal_scheduler.py

async def scan_disposable_my_numbers():
    """法定保存期間を経過したマイナンバーを検出し、廃棄候補リストを生成する。
    自動廃棄は行わない。管理者に通知して手動承認後に廃棄実行。"""

    # 1. 全アクティブなmy_numbersを取得
    # 2. 各レコードの最終利用日と保存期間を計算
    # 3. 廃棄可能日を過ぎているレコードを検出
    # 4. 管理者に通知（LINE WORKS / Slack / メール）
    # 5. 管理者が承認 → dispose_my_number() を実行
```

自動廃棄はしない。 必ず `my_number_admin` の手動承認を経て廃棄する（HITL）。

項目9: RLS・職務分離ポリシー設計

9.1 ロール定義

既存の2ロール（`admin`, `editor`）をバックオフィスモジュール用に拡張する。

ロール	説明	付与対象
`admin`	全権管理者。全テーブルの読み書き	社長・IT管理者
`hr_manager`	人事労務管理者。従業員・勤怠・給与の管理	人事部長・労務担当
`finance`	経理財務管理者。仕訳・請求・入金・銀行の管理	経理部長・財務担当
`my_number_admin`	マイナンバー管理者。番号の収集・復号・廃棄	特定個人情報取扱担当者（最小限）
`employee`	一般従業員。自分のデータのみ参照	全従業員
`auditor`	監査人。全テーブル読み取り専用（書き込み不可）	外部監査人・内部監査担当

ロールの付与ルール:
- 1ユーザーに複数ロールを付与可（例: `hr_manager` + `my_number_admin`）
- `my_number_admin` は最小限の人数に限定（推奨: 2名以下）
- ロールの付与・剥奪は `admin` のみ可。`audit_logs` に記録

usersテーブルの拡張:

```
-- 既存の role TEXT を roles TEXT[] に変更
ALTER TABLE users ADD COLUMN roles TEXT[] NOT NULL DEFAULT ARRAY['employee'];
-- マイグレーション時に既存データを移行:
-- role = 'admin' → roles = ['admin']
-- role = 'editor' → roles = ['employee']
```

9.2 RLSポリシー — テーブル別アクセス制御マトリクス

9.2.1 全体マトリクス

```
C = CREATE, R = READ, U = UPDATE, D = DELETE
(自) = 自分のレコードのみ
(復) = 復号権限者のみ（my_number_admin）
— = アクセス不可
```

テーブル	admin	hr_manager	finance	employee	auditor
employees（基本情報）	CRUD	CRUD	R	R(自)	R
employees（給与カラム）	R	CRUD	R	R(自)	R
employees（マイナンバー）	—	R(復)	—	—	—
payroll_runs	R	CRUD	CRUD	—	R
payroll_items	R	R	CRUD	R(自)	R
journal_entries	R	—	CRUD	—	R
journal_entry_lines	R	—	CRUD	—	R
bank_accounts	R	—	CRUD	—	R
bank_transactions	R	—	CRUD	—	R
timecards	R	R	—	CRUD(自)	R
timecard_approvals	R	CRUD	—	R(自)	R
invoices	R	—	CRUD	—	R
invoice_items	R	—	CRUD	—	R
purchase_orders	R	—	CRUD	—	R
inventory_items	R	—	CRUD	—	R
inventory_transactions	R	—	CRUD	—	R
approval_requests	R	CRUD	CRUD	R(自)	R
my_numbers	—	—(※)	—	—	—
my_number_access_log	R	—	—	—	R

※ `my_numbers` テーブルはRLSではなくアプリ層で制御（§8.3.3参照）

9.2.2 RLSポリシーSQL（主要テーブル）

```
-- ============================================================
-- employees テーブル
-- ============================================================
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;

-- テナント分離（全ロール共通の基盤ポリシー）
CREATE POLICY "employees_tenant" ON employees
    USING (company_id = (current_setting('app.company_id', true))::UUID);

-- admin: CRUD
CREATE POLICY "employees_admin_all" ON employees
    FOR ALL
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['admin']
    );

-- hr_manager: CRUD
CREATE POLICY "employees_hr_all" ON employees
    FOR ALL
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['hr_manager']
    );

-- finance: READ ONLY
CREATE POLICY "employees_finance_read" ON employees
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['finance']
    );

-- employee: 自分のレコードのみREAD
CREATE POLICY "employees_self_read" ON employees
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND user_id = (current_setting('app.user_id', true))::UUID
    );

-- auditor: READ ONLY
CREATE POLICY "employees_auditor_read" ON employees
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['auditor']
    );
```

```
-- ============================================================
-- timecards テーブル
-- ============================================================
ALTER TABLE timecards ENABLE ROW LEVEL SECURITY;

-- admin / hr_manager: READ
CREATE POLICY "timecards_admin_hr_read" ON timecards
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['admin',
'hr_manager']
    );

-- employee: 自分のレコードのみCRUD
CREATE POLICY "timecards_self_crud" ON timecards
    FOR ALL
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND employee_id = (current_setting('app.employee_id', true))::UUID
    );

-- ============================================================
-- journal_entries テーブル
-- ============================================================
ALTER TABLE journal_entries ENABLE ROW LEVEL SECURITY;

-- finance: CRUD
CREATE POLICY "journal_entries_finance_all" ON journal_entries
    FOR ALL
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['finance']
    );

-- admin / auditor: READ ONLY
CREATE POLICY "journal_entries_admin_auditor_read" ON journal_entries
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['admin',
'auditor']
    );

-- ============================================================
-- payroll_runs / payroll_items テーブル
-- ============================================================
ALTER TABLE payroll_runs ENABLE ROW LEVEL SECURITY;

-- hr_manager + finance: CRUD
CREATE POLICY "payroll_runs_hr_finance_all" ON payroll_runs
    FOR ALL
```

```
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['hr_manager',
'finance']
    );

-- admin / auditor: READ ONLY
CREATE POLICY "payroll_runs_admin_auditor_read" ON payroll_runs
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['admin',
'auditor']
    );

ALTER TABLE payroll_items ENABLE ROW LEVEL SECURITY;

-- finance: CRUD
CREATE POLICY "payroll_items_finance_all" ON payroll_items
    FOR ALL
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['finance']
    );

-- admin / hr_manager / auditor: READ ONLY
CREATE POLICY "payroll_items_read" ON payroll_items
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND (current_setting('app.user_roles', true))::TEXT[] && ARRAY['admin',
'hr_manager', 'auditor']
    );

-- employee: 自分のレコードのみREAD
CREATE POLICY "payroll_items_self_read" ON payroll_items
    FOR SELECT
    USING (
        company_id = (current_setting('app.company_id', true))::UUID
        AND employee_id = (current_setting('app.employee_id', true))::UUID
    );
```

9.3 職務分離（Separation of Duties）

9.3.1 分離すべきアクション対

#	アクション A	アクション B	リスク	分離レベル
SoD-1	給与計算の実行	給与振込の承認	架空従業員への振込	**必須**
SoD-2	仕訳入力	仕訳承認	粉飾決算・横領	**必須**
SoD-3	請求書作成	入金消込	架空請求・着服	**必須**
SoD-4	発注	検収	架空発注・キックバック	**必須**
SoD-5	従業員登録	給与設定	架空従業員+高額給与	**必須**
SoD-6	経費申請	経費承認	私的流用	推奨
SoD-7	マイナンバー収集	マイナンバー利用（帳票出力）	番号の目的外利用	推奨

9.3.2 approval_rules テーブル設計

```
CREATE TABLE approval_rules (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    company_id UUID NOT NULL REFERENCES companies(id),
    rule_code TEXT NOT NULL,                    -- 'SoD-1', 'SoD-2', etc.
    action_a TEXT NOT NULL,                     -- 'payroll_calculate'
    action_b TEXT NOT NULL,                     -- 'payroll_approve_transfer'
    enforcement TEXT NOT NULL DEFAULT 'hard'
        CHECK (enforcement IN ('hard', 'soft')),  -- hard=システム強制, soft=警告のみ
    is_active BOOLEAN NOT NULL DEFAULT true,
    description TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(company_id, rule_code)
);

-- デフォルトルールの投入（onboarding時）
INSERT INTO approval_rules (company_id, rule_code, action_a, action_b, enforcement,
description)
VALUES
    (:cid, 'SoD-1', 'payroll_calculate', 'payroll_approve_transfer', 'hard',
     '給与計算者と振込承認者は同一人物不可'),
    (:cid, 'SoD-2', 'journal_entry_create', 'journal_entry_approve', 'hard',
     '仕訳入力者と承認者は同一人物不可'),
    (:cid, 'SoD-3', 'invoice_create', 'payment_reconcile', 'hard',
     '請求書作成者と入金消込者は同一人物不可'),
    (:cid, 'SoD-4', 'purchase_order_create', 'purchase_order_inspect', 'hard',
     '発注者と検収者は同一人物不可'),
    (:cid, 'SoD-5', 'employee_register', 'salary_configure', 'hard',
     '従業員登録者と給与設定者は同一人物不可');
```

9.3.3 承認リクエストテーブル設計

```
CREATE TABLE approval_requests (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    company_id UUID NOT NULL REFERENCES companies(id),
    request_type TEXT NOT NULL,                -- 'payroll_approve_transfer' 等
    related_record_type TEXT NOT NULL,         -- 'payroll_runs'
    related_record_id UUID NOT NULL,
    requested_by UUID NOT NULL REFERENCES users(id),
    status TEXT NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'approved', 'rejected', 'cancelled')),
    approved_by UUID REFERENCES users(id),
    approved_at TIMESTAMPTZ,
    rejection_reason TEXT,
    details JSONB,                             -- 金額・対象期間等の要約
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_approval_requests_company ON approval_requests (company_id, status,
created_at DESC);
CREATE INDEX idx_approval_requests_approver ON approval_requests (approved_by, status);
```

9.3.4 職務分離チェックの実装方針

```
# security/separation_of_duties.py

async def check_sod_constraint(
    company_id: UUID,
    action: str,
    user_id: UUID,
    related_record_id: UUID,
) -> None:
    """職務分離制約をチェック。違反時は SoDViolationError を送出。"""

    # 1. approval_rules から、action を action_b として持つルールを検索
    rules = await _get_sod_rules(company_id, action_b=action)

    for rule in rules:
        # 2. 対象レコードに対して action_a を実行したユーザーを特定
        action_a_executor = await _get_action_executor(
            company_id, rule.action_a, related_record_id
        )

        # 3. 同一人物チェック
        if action_a_executor == user_id:
            if rule.enforcement == "hard":
                raise SoDViolationError(
                    f"職務分離違反: {rule.description}。"
                    f"別の担当者に承認を依頼してください。"
                )
            else:
                # soft: 警告ログを記録し、続行を許可
                await _log_sod_warning(company_id, rule, user_id, related_record_id)
```

呼び出し例（給与振込承認時）：

```
@router.post("/payroll/runs/{run_id}/approve-transfer")
async def approve_payroll_transfer(
    run_id: UUID,
    user: JWTClaims = Depends(require_role("finance")),
):
    # 職務分離チェック
    await check_sod_constraint(
        company_id=user.company_id,
        action="payroll_approve_transfer",
        user_id=user.user_id,
        related_record_id=run_id,
    )
    # 承認処理...
```

9.3.5 少人数企業への配慮

従業員10名以下の企業では、職務分離が物理的に不可能な場合がある。

従業員数	対応
1〜3名	全SoDルールを `soft`（警告のみ）に設定。操作はログに残る
4〜10名	SoD-1, SoD-2 のみ `hard`。他は `soft`
11名以上	全SoDルールを `hard` に設定（推奨）

オンボーディング時に従業員数に応じてデフォルト設定を投入する。`admin` はいつでも変更可能。

9.4 PII暗号化範囲の拡大

9.4.1 暗号化対象フィールド一覧

テーブル	カラム	分類	暗号化鍵	復号タイミング
`my_numbers.encrypted_number`	マイナンバー	特定個人情報	`MY_NUMBER_ENCRYPTION_KEY`	帳票出力時のみ
`employees.phone`	電話番号	個人情報	`ENCRYPTION_KEY`	画面表示時
`employees.address`	住所	個人情報	`ENCRYPTION_KEY`	画面表示時
`employees.email_personal`	個人メールアドレス	個人情報	`ENCRYPTION_KEY`	画面表示時
`bank_accounts.account_number`	口座番号	金融情報	`FINANCIAL_ENCRYPTION_KEY`	振込処理時
`bank_accounts.account_holder`	口座名義	金融情報	`FINANCIAL_ENCRYPTION_KEY`	振込処理時
`dependents.name`	扶養家族氏名	個人情報	`ENCRYPTION_KEY`	画面表示時
`dependents.birth_date`	扶養家族生年月日	個人情報	`ENCRYPTION_KEY`	画面表示時
`dependents.my_number`	扶養家族マイナンバー	特定個人情報	`MY_NUMBER_ENCRYPTION_KEY`	帳票出力時のみ

9.4.2 ハッシュインデックスによる検索

暗号化フィールドは平文でDB検索できない。検索が必要なフィールドにはハッシュインデックスを用意する。

```
-- employees テーブルに検索用ハッシュカラムを追加
ALTER TABLE employees ADD COLUMN phone_hash TEXT;
ALTER TABLE employees ADD COLUMN address_hash TEXT;

-- ハッシュインデックス
CREATE INDEX idx_employees_phone_hash ON employees (company_id, phone_hash);
```

ハッシュ生成:

```
import hashlib

def compute_search_hash(value: str, salt: str) -> str:
    """検索用のSHA-256ハッシュを生成。"""
    return hashlib.sha256(f"{salt}:{value}".encode()).hexdigest()
```

検索フロー:

1. ユーザーが電話番号で検索
2. 入力値をハッシュ化
3. `phone_hash` カラムで完全一致検索
4. ヒットしたレコードの暗号化フィールドを復号して表示

注意: 部分一致検索（前方一致等）はハッシュでは不可能。住所の部分検索が必要な場合は、都道府県・市区町村を別カラム（平文）で持つ設計にする。

9.4.3 暗号化・復号の共通ミドルウェア

```
# security/pii_middleware.py

# 暗号化対象フィールドの定義
ENCRYPTED_FIELDS = {
    "employees": {
        "phone": {"key": KeyPurpose.GENERAL_PII, "searchable": True},
        "address": {"key": KeyPurpose.GENERAL_PII, "searchable": True},
        "email_personal": {"key": KeyPurpose.GENERAL_PII, "searchable": False},
    },
    "bank_accounts": {
        "account_number": {"key": KeyPurpose.FINANCIAL, "searchable": False},
        "account_holder": {"key": KeyPurpose.FINANCIAL, "searchable": False},
    },
    "dependents": {
        "name": {"key": KeyPurpose.GENERAL_PII, "searchable": False},
        "birth_date": {"key": KeyPurpose.GENERAL_PII, "searchable": False},
        "my_number": {"key": KeyPurpose.MY_NUMBER, "searchable": False},
    },
}

async def encrypt_record(table: str, record: dict) -> dict:
    """レコード内の暗号化対象フィールドを暗号化。DB保存前に呼ぶ。"""
    fields = ENCRYPTED_FIELDS.get(table, {})
    for field_name, config in fields.items():
        if field_name in record and record[field_name] is not None:
            plaintext = record[field_name]
            record[field_name] = encrypt_field(plaintext, purpose=config["key"])
            if config["searchable"]:
                record[f"{field_name}_hash"] = compute_search_hash(plaintext, HASH_SALT)
    return record

async def decrypt_record(table: str, record: dict, allowed_fields: list[str] = None) ->
dict:
    """レコード内の暗号化フィールドを復号。画面表示前に呼ぶ。"""
    fields = ENCRYPTED_FIELDS.get(table, {})
    for field_name, config in fields.items():
        if field_name in record and record[field_name] is not None:
            if allowed_fields and field_name not in allowed_fields:
                record[field_name] = "***"  # 復号権限なし
                continue
            record[field_name] = decrypt_field(record[field_name], purpose=config["key"])
    return record
```

項目10: フロントエンド7モジュールUI設計概要

> 全ページは `UI_RULES.md` を厳守する。
> 用語: BPO→業務自動化、パイプライン→業務フロー、etc.
> 対象ユーザー: IT・AIリテラシーのない中小企業の経営者・現場スタッフ
> モバイルファーストで設計。

10.0 モジュール全体マップ

```
/accounting     会計モジュール          8ページ
/hr             人事モジュール          8ページ
/payroll        給与モジュール          7ページ
/attendance     勤怠モジュール          6ページ
/banking        銀行モジュール          6ページ
/inventory      在庫モジュール          6ページ
/executive      経営ダッシュボード     9ページ
                                      ──────
                                      合計 50ページ
```

権限と表示制御:

モジュール	admin	hr_manager	finance	employee	auditor
会計	全機能	—	全機能	—	閲覧のみ
人事	全機能	全機能	—	自分のみ閲覧	閲覧のみ
給与	閲覧	計算・明細	承認・振込	自分の明細のみ	閲覧のみ
勤怠	全機能	承認	—	自分の打刻・申請	閲覧のみ
銀行	閲覧	—	全機能	—	閲覧のみ
在庫	全機能	—	全機能	—	閲覧のみ
経営ダッシュボード	全機能	—	部分閲覧	—	閲覧のみ

10.1 会計モジュール（/accounting）

ページ一覧

#	URL	ページ名	主要機能	権限
A-1	`/accounting`	会計ホーム	月次サマリー・未処理仕訳件数・キャッシュフロー概要	admin, finance, auditor
A-2	`/accounting/journals`	仕訳一覧	仕訳の検索・フィルタ・一覧表示。ステータス別タブ（下書き/申請中/承認済み）	admin, finance, auditor
A-3	`/accounting/journals/new`	仕訳入力	新規仕訳の作成。借方・貸方の入力。勘定科目の選択。AI仕訳提案	finance
A-4	`/accounting/journals/[id]`	仕訳詳細	仕訳の詳細表示・編集・承認・却下。証憑添付	admin, finance, auditor(閲覧のみ)
A-5	`/accounting/chart-of-accounts`	勘定科目マスタ	勘定科目の一覧・追加・編集。業界テンプレートからの初期投入	admin, finance
A-6	`/accounting/reports/pl`	損益計算書	月次/四半期/年次の損益計算書。前年同期比較	admin, finance, auditor
A-7	`/accounting/reports/bs`	貸借対照表	月次/四半期/年次の貸借対照表。推移分析	admin, finance, auditor
A-8	`/accounting/reports/cf`	キャッシュフロー計算書	営業/投資/財務の三区分表示。資金繰り予測	admin, finance, auditor

主要コンポーネント

コンポーネント	使用ページ	説明
`JournalEntryForm`	A-3, A-4	借方・貸方の動的行追加。勘定科目セレクト。貸借一致バリデーション
`JournalTable`	A-2	仕訳一覧テーブル。ステータスバッジ。スマホではカード表示に切替
`AccountSelector`	A-3, A-4	勘定科目の検索・選択コンボボックス。コード+名称でフィルタ
`ApprovalButtons`	A-4	「この仕訳を承認する」「差し戻す」ボタン。SoD-2制約を表示
`FinancialReportChart`	A-6, A-7, A-8	棒グラフ・折れ線グラフ。月次推移。前年同期比較
`AIJournalSuggestion`	A-3	AI が証憑画像から仕訳を提案。確度バッジ付き

データフロー

```
[A-3: 仕訳入力]
  ユーザー入力 → POST /api/v1/accounting/journals
    → journal_entries INSERT (status='draft')
    → journal_entry_lines INSERT (複数行)
    → audit_logs INSERT

[A-4: 仕訳承認]
  承認ボタン → PATCH /api/v1/accounting/journals/{id}/approve
    → SoD-2チェック (security/separation_of_duties.py)
    → journal_entries UPDATE (status='approved')
    → approval_requests INSERT
    → audit_logs INSERT

[A-6: 損益計算書]
  ページ表示 → GET /api/v1/accounting/reports/pl?period=2026-03
    → journal_entries + journal_entry_lines を集計
    → 勘定科目マスタと結合
    → レスポンス (科目別集計JSON)
```

10.2 人事モジュール（/hr）

ページ一覧

#	URL	ページ名	主要機能	権限
H-1	`/hr`	人事ホーム	在籍者数・入退社予定・組織図概要	admin, hr_manager, auditor
H-2	`/hr/employees`	従業員一覧	従業員の検索・フィルタ。部署・雇用形態別	admin, hr_manager, auditor
H-3	`/hr/employees/new`	従業員登録	新規従業員の登録。基本情報・雇用形態・部署	admin, hr_manager
H-4	`/hr/employees/[id]`	従業員詳細	基本情報・家族情報・給与情報・マイナンバー（マスク）。タブ切替	admin, hr_manager, employee(自分)
H-5	`/hr/employees/[id]/my-number`	マイナンバー管理	マイナンバーの登録・マスク表示・廃棄。§8の全仕様を実装	my_number_admin
H-6	`/hr/dependents`	扶養家族一覧	扶養家族の一覧。年末調整用データ確認	admin, hr_manager
H-7	`/hr/organization`	組織図	部署ツリー表示。部署の追加・編集・統廃合	admin, hr_manager
H-8	`/hr/reports`	人事レポート	在籍推移・離職率・平均勤続年数。グラフ表示	admin, hr_manager, auditor

主要コンポーネント

コンポーネント	使用ページ	説明
`EmployeeForm`	H-3, H-4	従業員情報入力フォーム。PII項目は保存時に自動暗号化
`EmployeeTable`	H-2	従業員一覧。部署・雇用形態フィルタ。スマホはカード表示
`EmployeeDetailTabs`	H-4	タブ: 基本情報 / 家族情報 / 給与情報 / 履歴。ロールで表示タブを制御
`MyNumberForm`	H-5	§8.2.2の入力フォーム仕様を完全実装。password入力+チェックデジット検証
`MyNumberMaskedDisplay`	H-4	`****-****-1234` 形式。「登録済み」/「未登録」バッジ
`OrgChart`	H-7	ツリー図。D3.jsまたはReactFlowで描画
`DependentForm`	H-4, H-6	扶養家族の追加・編集。氏名・生年月日は暗号化保存

データフロー

```
[H-3: 従業員登録]
  フォーム送信 → POST /api/v1/hr/employees
    → pii_middleware.encrypt_record('employees', data)
    → employees INSERT
    → audit_logs INSERT
    ※ SoD-5: 従業員登録者 ≠ 給与設定者 を記録

[H-5: マイナンバー登録]
  フォーム送信 → POST /api/v1/hr/employees/{id}/my-number
    → require_role('my_number_admin')
    → チェックデジット検証
    → my_number_consents INSERT
    → encrypt_field(number, purpose=KeyPurpose.MY_NUMBER)
    → my_numbers INSERT
    → my_number_access_log INSERT (access_type='collect')
    → audit_logs INSERT

[H-4: 従業員詳細表示]
  ページ表示 → GET /api/v1/hr/employees/{id}
    → employees SELECT
    → pii_middleware.decrypt_record('employees', record, allowed_fields)
    → ロール判定: my_number表示可否
    → レスポンス（暗号化フィールドは復号 or マスク）
```

10.3 給与モジュール（/payroll）

ページ一覧

#	URL	ページ名	主要機能	権限
P-1	`/payroll`	給与ホーム	今月の給与処理状況・締切日・未処理件数	admin, hr_manager, finance, auditor
P-2	`/payroll/runs`	給与計算一覧	月次給与計算の一覧。ステータス別（計算中/確認待ち/承認済み/振込済み）	admin, hr_manager, finance, auditor
P-3	`/payroll/runs/new`	給与計算実行	対象月選択 → 勤怠データ取込 → 自動計算実行	hr_manager
P-4	`/payroll/runs/[id]`	給与計算詳細	従業員別の給与明細一覧。修正・承認	admin, hr_manager, finance
P-5	`/payroll/runs/[id]/approve`	給与振込承認	振込データの確認と承認。SoD-1制約を表示	finance
P-6	`/payroll/slips/mine`	自分の給与明細	ログインユーザー自身の給与明細。月選択。PDF出力	employee
P-7	`/payroll/settings`	給与設定	社会保険料率・所得税テーブル・手当マスタ・控除マスタ	admin, hr_manager

主要コンポーネント

コンポーネント	使用ページ	説明
`PayrollRunTable`	P-2	給与計算一覧。ステータスバッジ。対象人数・総額表示
`PayrollCalculationWizard`	P-3	ステップ: 対象月選択 → 勤怠データ確認 → 計算実行 → 結果確認
`PayrollDetailTable`	P-4	従業員別明細テーブル。基本給・手当・控除・差引支給額
`TransferApprovalPanel`	P-5	振込先一覧+合計金額。SoD-1チェック結果表示。「振込を承認する」ボタン
`PayslipViewer`	P-6	給与明細のカード表示。月選択セレクト。PDF出力ボタン
`DeductionSettingsForm`	P-7	社会保険料率・標準報酬月額テーブル設定

データフロー

```
[P-3: 給与計算実行]
  ウィザード完了 → POST /api/v1/payroll/runs
    → timecards から勤怠データ取込
    → employees から基本給・手当マスタ取込
    → 給与計算エンジン実行（社保・所得税・住民税自動計算）
    → payroll_runs INSERT (status='calculated')
    → payroll_items INSERT (従業員人数分)
    → audit_logs INSERT

[P-5: 給与振込承認]
  承認ボタン → POST /api/v1/payroll/runs/{id}/approve-transfer
    → check_sod_constraint('payroll_approve_transfer', user_id, run_id)
    → payroll_runs UPDATE (status='transfer_approved')
    → approval_requests INSERT
    → 全銀フォーマットデータ生成（bank_transactions連携）
    → audit_logs INSERT

[P-6: 自分の給与明細]
  ページ表示 → GET /api/v1/payroll/slips/mine?month=2026-03
    → RLS: employee_id = current_user.employee_id
    → payroll_items SELECT
    → レスポンス（自分の明細のみ）
```

10.4 勤怠モジュール（/attendance）

ページ一覧

#	URL	ページ名	主要機能	権限
T-1	`/attendance`	勤怠ホーム	今日の出退勤ボタン・今月のサマリー・残業時間	全ロール
T-2	`/attendance/timecard`	自分の勤怠表	月次勤怠一覧。日別の出退勤時刻・労働時間。修正申請	employee
T-3	`/attendance/timecard/edit/[date]`	勤怠修正申請	打刻忘れ・修正の申請フォーム。理由入力必須	employee
T-4	`/attendance/approvals`	勤怠承認	部下の勤怠修正申請・残業申請の承認/却下	hr_manager, admin
T-5	`/attendance/team`	チーム勤怠一覧	部署全員の勤怠状況。遅刻・欠勤・残業アラート	hr_manager, admin, auditor
T-6	`/attendance/reports`	勤怠レポート	月次集計。残業時間ランキング。36協定チェック	hr_manager, admin, auditor

主要コンポーネント

コンポーネント	使用ページ	説明
`ClockInOutButton`	T-1	大きなタップボタン（出勤/退勤切替）。現在時刻表示。GPS取得（オプション）
`TimecardCalendar`	T-2	カレンダー形式の月次勤怠表。色分け（出勤/休日/有給/遅刻）
`TimecardEditForm`	T-3	出退勤時刻の修正入力。理由テキストエリア。証跡添付
`ApprovalQueue`	T-4	未承認申請のキュー表示。一括承認ボタン
`TeamAttendanceGrid`	T-5	部署×日付のグリッド表示。異常値ハイライト
`OvertimeAlert`	T-5, T-6	36協定の上限（月45h/年360h）に近い従業員を警告バッジで表示

データフロー

```
[T-1: 出退勤打刻]
  出勤ボタン → POST /api/v1/attendance/clock-in
    → timecards INSERT/UPDATE (clock_in = NOW())
    → audit_logs INSERT

  退勤ボタン → POST /api/v1/attendance/clock-out
    → timecards UPDATE (clock_out = NOW(), work_hours = 計算)
    → audit_logs INSERT

[T-4: 勤怠承認]
  承認ボタン → PATCH /api/v1/attendance/approvals/{id}
    → timecard_approvals UPDATE (status='approved')
    → timecards UPDATE（修正値を反映）
    → audit_logs INSERT

[T-6: 勤怠レポート]
  ページ表示 → GET /api/v1/attendance/reports?month=2026-03
    → timecards 集計（部署別・従業員別）
    → 36協定チェック（月45h/年360h超過者リスト）
    → レスポンス（集計JSON + アラート対象者リスト）
```

10.5 銀行モジュール（/banking）

ページ一覧

#	URL	ページ名	主要機能	権限
B-1	`/banking`	銀行ホーム	口座残高サマリー・今月の入出金推移・資金繰り予測	admin, finance, auditor
B-2	`/banking/accounts`	口座一覧	登録口座の一覧・残高表示。口座追加・編集	admin, finance
B-3	`/banking/transactions`	入出金明細	全口座の入出金明細。フィルタ（口座/期間/金額/摘要）。仕訳連携ステータス	admin, finance, auditor
B-4	`/banking/reconciliation`	消込	入金と請求書の消込。AI消込提案。未消込一覧	finance
B-5	`/banking/transfers`	振込管理	振込データの作成・承認・実行。全銀フォーマット出力	finance
B-6	`/banking/cashflow`	資金繰り予測	今後3ヶ月の入出金予測。グラフ表示。資金ショートアラート	admin, finance, auditor

主要コンポーネント

コンポーネント	使用ページ	説明
`AccountBalanceCards`	B-1	口座ごとのカード。残高・前月比。スマホ横スクロール
`TransactionTable`	B-3	入出金明細テーブル。仕訳連携バッジ（「仕訳済み」/「未連携」）
`ReconciliationPanel`	B-4	左: 入金明細、右: 請求書。ドラッグ＆ドロップ or AI提案で紐付け。SoD-3制約表示
`TransferForm`	B-5	振込先選択・金額入力。全銀フォーマットプレビュー
`CashflowChart`	B-6	折れ線グラフ（実績+予測）。危険ライン表示
`BankAccountForm`	B-2	口座情報入力。口座番号は暗号化保存。マスク表示

データフロー

```
[B-4: 入金消込]
  AI消込提案取得 → GET /api/v1/banking/reconciliation/suggestions
    → bank_transactions (入金) + invoices (請求書) を照合
    → AI が金額・日付・摘要から対応候補を提案
    → レスポンス (候補ペアリスト + 確度)

  消込確定 → POST /api/v1/banking/reconciliation
    → check_sod_constraint('payment_reconcile', user_id, invoice_id)
    → bank_transactions UPDATE (reconciled=true)
    → invoices UPDATE (status='paid')
    → journal_entries INSERT (入金仕訳自動生成)
    → audit_logs INSERT

[B-5: 振込承認]
  承認 → POST /api/v1/banking/transfers/{id}/approve
    → approval_requests INSERT
    → 全銀フォーマットファイル生成
    → bank_transactions INSERT (type='transfer')
    → audit_logs INSERT
```

10.6 在庫モジュール（/inventory）

ページ一覧

#	URL	ページ名	主要機能	権限
I-1	`/inventory`	在庫ホーム	在庫金額サマリー・発注アラート・回転率	admin, finance, auditor
I-2	`/inventory/items`	商品・資材一覧	商品マスタの一覧。在庫数・単価・安全在庫。フィルタ検索	admin, finance
I-3	`/inventory/items/[id]`	商品詳細	在庫推移グラフ・入出庫履歴・発注点設定	admin, finance
I-4	`/inventory/transactions`	入出庫履歴	入庫・出庫・調整の履歴一覧。伝票番号・理由	admin, finance, auditor
I-5	`/inventory/purchase-orders`	発注管理	発注書の作成・承認・納品確認。SoD-4制約	admin, finance
I-6	`/inventory/reports`	在庫レポート	在庫回転率・滞留在庫・ABC分析。グラフ表示	admin, finance, auditor

主要コンポーネント

コンポーネント	使用ページ	説明
`InventoryDashboardCards`	I-1	在庫金額・アイテム数・発注アラート件数のカード
`ItemTable`	I-2	商品一覧テーブル。在庫数が安全在庫以下は赤ハイライト
`StockChart`	I-3	在庫推移の折れ線グラフ。安全在庫ライン表示
`TransactionHistory`	I-3, I-4	入出庫履歴テーブル。タイプ別バッジ（入庫/出庫/調整）
`PurchaseOrderForm`	I-5	発注書作成フォーム。品目・数量・納期・仕入先。SoD-4制約表示
`ABCAnalysisChart`	I-6	パレート図。A/B/Cランク色分け

データフロー

```
[I-5: 発注]
  発注書作成 → POST /api/v1/inventory/purchase-orders
    → purchase_orders INSERT (status='draft')
    → audit_logs INSERT

  検収（納品確認）→ POST /api/v1/inventory/purchase-orders/{id}/inspect
    → check_sod_constraint('purchase_order_inspect', user_id, po_id)
    → purchase_orders UPDATE (status='inspected')
    → inventory_items UPDATE (quantity += received_qty)
    → inventory_transactions INSERT (type='receiving')
    → audit_logs INSERT

[I-6: ABC分析]
  ページ表示 → GET /api/v1/inventory/reports/abc
    → inventory_items + inventory_transactions 集計
    → 出庫金額でランク付け（A: 上位70%, B: 次の20%, C: 残り10%）
    → レスポンス（ランク付きアイテムリスト + パレート図データ）
```

10.7 経営ダッシュボード（/executive）

ページ一覧

#	URL	ページ名	主要機能	権限
E-1	`/executive`	経営ダッシュボード	全モジュールのKPI統合ビュー。売上・利益・人件費率・資金繰り	admin
E-2	`/executive/pl-analysis`	損益分析	月次推移・前年比・予実比較。AIによる異常値検知	admin, finance, auditor
E-3	`/executive/cashflow`	キャッシュフロー分析	全口座統合の資金繰り。3ヶ月予測。シナリオシミュレーション	admin, finance
E-4	`/executive/labor-cost`	人件費分析	部署別人件費・売上人件費率・残業コスト。推移グラフ	admin, hr_manager
E-5	`/executive/kpi`	KPIトラッカー	自社設定のKPI一覧。達成率・推移。目標値の設定	admin
E-6	`/executive/alerts`	アラート一覧	全モジュールの異常・警告を統合表示。優先度順	admin
E-7	`/executive/what-if`	What-ifシミュレーション	「従業員を3名増やしたら？」「売上が20%減ったら？」のシミュレーション	admin
E-8	`/executive/reports`	月次経営レポート	全モジュールのサマリーをPDF出力。取締役会資料	admin, auditor
E-9	`/executive/audit-log`	監査ログ	全操作の監査ログ検索・表示。フィルタ（操作者/期間/種別）	admin, auditor

主要コンポーネント

コンポーネント	使用ページ	説明
`KPICard`	E-1, E-5	KPI値 + 前月比 + 推移スパークライン。赤/黄/緑のステータス
`ExecutivePLChart`	E-1, E-2	売上・原価・販管費・利益の月次推移チャート
`CashflowForecast`	E-3	実績+予測の折れ線グラフ。シナリオ切替（楽観/標準/悲観）
`LaborCostBreakdown`	E-4	部署別人件費の積み上げ棒グラフ。売上人件費率の推移
`AlertFeed`	E-6	アラートのタイムライン表示。重要度バッジ。対処ボタン
`WhatIfSimulator`	E-7	パラメータスライダー（従業員数/売上/原価率等）→ リアルタイムで損益・キャッシュフロー変化を表示
`AuditLogTable`	E-9	監査ログの検索テーブル。操作者・操作種別・対象・日時。詳細展開
`MonthlyReportGenerator`	E-8	レポート期間選択 → 「レポートを生成する」→ PDF出力

データフロー

```
[E-1: 経営ダッシュボード]
  ページ表示 → GET /api/v1/executive/dashboard
    → 並列API呼び出し:
      GET /api/v1/accounting/reports/pl?period=current (損益サマリー)
      GET /api/v1/banking/cashflow/summary (資金繰りサマリー)
      GET /api/v1/payroll/runs/latest/summary (人件費サマリー)
      GET /api/v1/executive/alerts/count (アラート件数)
    → レスポンス（統合KPIデータ）

[E-7: What-ifシミュレーション]
  パラメータ変更 → POST /api/v1/executive/what-if
    → company_state_snapshots から最新状態取得
    → パラメータを変更して損益・CFを再計算（LLM不使用、数式ベース）
    → レスポンス（シミュレーション結果JSON）

[E-9: 監査ログ]
  検索 → GET /api/v1/audit-logs?user_id=xxx&action=xxx&from=xxx&to=xxx
    → audit_logs SELECT（RLSでテナント分離）
    → レスポンス（ログ一覧 + ページネーション）
```

10.8 共通UIパターン

10.8.1 承認UIパターン（全モジュール共通）

仕訳承認・給与振込承認・発注検収等、承認が必要な操作は共通コンポーネントを使う。

```
┌──────────────────────────────────────────────────┐
│  給与振込を承認する                                 │
│                                                    │
│  対象: 2026年3月分 給与                             │
│  対象人数: 25名                                     │
│  振込合計: ¥8,750,000                               │
│                                                    │
│  ⚠ 給与計算の実行者（山田太郎さん）とは              │
│    別の担当者が承認する必要があります                 │
│                                                    │
│  [キャンセル]  [振込を承認する]                      │
└──────────────────────────────────────────────────┘
```

10.8.2 暗号化フィールドの表示パターン

```
[通常表示（復号権限あり）]
  電話番号: 090-1234-5678
  住所: 東京都渋谷区...

[復号権限なし]
  電話番号: ***（閲覧権限がありません）
  住所: ***（閲覧権限がありません）

[マイナンバー（my_number_admin）]
  マイナンバー: ****-****-1234

[マイナンバー（その他ロール）]
  → フィールド自体を非表示
```

10.8.3 SoD制約の表示パターン

```
[制約に抵触する場合]
  ⚠ この操作は実行できません
  理由: 「仕訳入力者と承認者は同一人物不可」のルールにより、
  仕訳を入力した本人は承認できません。
  別の経理担当者に承認を依頼してください。

  [承認依頼を送る]  ← 別ユーザーに通知を送るボタン

[少人数企業（soft制約）の場合]
  ⚠ 通常は別の担当者が承認しますが、
  少人数のため本人による承認を許可しています。
  この操作はログに記録されます。

  [承認する（本人承認）]
```

10.8.4 新規テーブル一覧（マイグレーション対象）

本設計で新規に必要なテーブルをまとめる。

#	テーブル名	用途	RLS	備考
1	`employees`	従業員マスタ	テナント+ロール	PII暗号化カラムあり
2	`dependents`	扶養家族	テナント	PII暗号化カラムあり
3	`my_numbers`	マイナンバー保管	アプリ層制御	§8.3参照
4	`my_number_consents`	マイナンバー同意記録	テナント	INSERT ONLY推奨
5	`my_number_identity_docs`	本人確認書類記録	テナント	
6	`my_number_access_log`	マイナンバーアクセスログ	INSERT ONLY	永久保存
7	`my_number_disposal_log`	マイナンバー廃棄ログ	INSERT ONLY	永久保存
8	`payroll_runs`	給与計算バッチ	テナント+ロール	
9	`payroll_items`	給与明細行	テナント+ロール+自分	
10	`timecards`	勤怠打刻	テナント+自分	
11	`timecard_approvals`	勤怠承認	テナント+ロール	
12	`journal_entries`	仕訳ヘッダ	テナント+ロール	
13	`journal_entry_lines`	仕訳明細行	テナント+ロール	
14	`chart_of_accounts`	勘定科目マスタ	テナント	業界テンプレート初期投入
15	`bank_accounts`	銀行口座	テナント+ロール	口座番号暗号化
16	`bank_transactions`	入出金明細	テナント+ロール	
17	`invoices`	請求書	テナント+ロール	
18	`invoice_items`	請求書明細行	テナント	
19	`purchase_orders`	発注書	テナント+ロール	
20	`purchase_order_items`	発注書明細行	テナント	
21	`inventory_items`	商品・資材マスタ	テナント	
22	`inventory_transactions`	入出庫履歴	テナント	
23	`approval_rules`	職務分離ルール	テナント	SoD制約定義
24	`approval_requests`	承認リクエスト	テナント+ロール	
25	`kpi_definitions`	KPI定義マスタ	テナント	経営ダッシュボード用

#	テーブル名	用途	RLS	備考
26	`kpi_values`	KPI実績値	テナント	月次自動集計

10.9 API エンドポイント一覧（ルーター別）

ルーターファイル	prefix	主要エンドポイント数
`routers/accounting.py`	`/api/v1/accounting`	12
`routers/hr.py`	`/api/v1/hr`	14
`routers/payroll_mgmt.py`	`/api/v1/payroll`	10
`routers/attendance.py`	`/api/v1/attendance`	10
`routers/banking.py`	`/api/v1/banking`	12
`routers/inventory_mgmt.py`	`/api/v1/inventory`	10
`routers/executive.py`	`/api/v1/executive`	12

合計: 約80エンドポイント

命名規則（既存ルーターとの衝突回避）:
- 給与: `payroll_mgmt.py`（既存の `payroll` と区別）
- 在庫: `inventory_mgmt.py`（将来の `inventory` BPOと区別）
- 勤怠: `attendance.py`（新規）

付録A: 実装優先順位

優先度	モジュール	理由
P0	勤怠（T-1〜T-6）	全従業員が毎日使う。利用頻度最高
P0	給与（P-1〜P-7）	毎月の締切がある。勤怠と密結合
P1	人事（H-1〜H-8）	従業員マスタは全モジュールの基盤
P1	会計（A-1〜A-8）	給与→仕訳の連携に必要
P2	銀行（B-1〜B-6）	振込・消込は会計モジュール稼働後
P2	在庫（I-1〜I-6）	製造業・卸売業のみ。業界依存
P3	経営ダッシュボード（E-1〜E-9）	他モジュールのデータが揃ってから

付録B: マイグレーションファイル番号

本設計のマイグレーションは以下の番号で作成する:

```
024_backoffice_employees.sql          -- employees, dependents
025_backoffice_my_number.sql          -- my_numbers, my_number_consents,
my_number_identity_docs,
                                      -- my_number_access_log, my_number_disposal_log
026_backoffice_attendance.sql         -- timecards, timecard_approvals
027_backoffice_payroll.sql            -- payroll_runs, payroll_items
028_backoffice_accounting.sql         -- journal_entries, journal_entry_lines,
chart_of_accounts
029_backoffice_banking.sql            -- bank_accounts, bank_transactions, invoices,
invoice_items
030_backoffice_inventory.sql          -- inventory_items, inventory_transactions,
                                      -- purchase_orders, purchase_order_items
031_backoffice_approval.sql           -- approval_rules, approval_requests
032_backoffice_executive.sql          -- kpi_definitions, kpi_values
033_backoffice_rls_policies.sql       -- 全テーブルのRLSポリシー一括定義
034_backoffice_indexes.sql            -- 全テーブルのインデックス一括定義
```